

MISA: Minimal Instruction Set Architecture

Jonathan Uhler

2026-05-01

Table of contents

Preface	4
1 Instruction Set Architecture	5
1.1 General Purpose Registers	5
1.2 Control and Status Registers	6
1.2.1 SADDR	6
1.2.2 RADDR	7
1.2.3 FLAGS	7
1.2.4 CAUSE	8
1.2.5 EXTNS	9
1.3 Data Formats and Alignments	10
1.4 Memory	10
1.4.1 Required Memory Regions	10
1.4.2 Recommended Memory Map	11
1.4.3 Stack Conventions	12
1.5 Reset	12
1.6 Faults	12
1.7 Exceptions and Interrupts	13
1.8 Base Instructions	13
1.8.1 HALT	13
1.8.2 ADD	14
1.8.3 ADC	14
1.8.4 SUB	14
1.8.5 SBB	15
1.8.6 AND	15
1.8.7 OR	15
1.8.8 XOR	16
1.8.9 RRC	16
1.8.10 SET	16
1.8.11 LD	17
1.8.12 ST	17
1.8.13 RSR	17
1.8.14 WSR	18
1.8.15 JAL	18
1.8.16 JMP	18

1.9	Pseudo Instructions	19
1.9.1	ADD2	19
1.9.2	AND2	19
1.9.3	CALL	19
1.9.4	CLR	19
1.9.5	CMP	20
1.9.6	GOTO	20
1.9.7	JALI	20
1.9.8	JMPI	20
1.9.9	MOV	20
1.9.10	MOV2	21
1.9.11	NOP	21
1.9.12	OR2	21
1.9.13	POP	21
1.9.14	POP2	22
1.9.15	PUSH	22
1.9.16	PUSH2	22
1.9.17	RET	22
1.9.18	RRC2	23
1.9.19	SET2	23
1.9.20	SUB2	23
1.9.21	XOR2	23
1.10	Architecture Extensions	23
1.10.1	Dynamic Hardware Extension	24
1.10.2	System Call Extension	24
1.10.3	Privilege Extension	26
2	Toolchain	28
2.1	Assembler	28
2.1.1	Assembler Pipeline	28
2.1.2	Assembler Semantics	29
2.1.3	MISA Linkable Format	30
2.2	Linker	33
2.3	Other Tools	33
3	Simulator And Hardware Reference	34
3.1	Functional Simulator	34
3.1.1	Loading And Reset	34
3.1.2	Instruction Execution	34
3.1.3	Address Space	35
3.2	Interactive Debugger	35
3.3	Hardware Reference	36

Preface

Minimal Instruction Set Architecture (MISA) is a limited instruction set and computer architecture designed for educational purposes. The ultimate design goal is to apply a large set of the knowledge from the average computer science undergraduate degree into one project that includes:

- Computer architecture
- Computer system design
- Logic design and computer engineering
- Data structures and algorithms
- Programming language theory
- Computability theory
- Maybe some basics on operating systems
- And various mathematical concepts

The core component of MISA is the namesake minimal instruction set architecture itself. While unlikely, a goal for the project is to tape out a very simple (i.e. single-cycle) processor that uses the MISA architecture. Given that, the ISA is intended to be as limited as possible, hopefully in a way that optimizes for the chip area constraints enforced by hobbyist tapeout programs. The ISA from a programmer's view is fully defined in Chapter 1 of this document.

The MISA project also includes a toolchain for creating programs to run on the MISA architecture. This includes an assembler, linker, and some object dumping/analysis tools. Some notes on the software design of these tools, in addition to their usage, are available in Chapter 2.

A software simulator is also available as the functional reference for the architecture. The intent of the simulator is to provide a full reference implementation for the functionality of the MISA instructions, so it defines how a conforming processor executes a program. Notes on the simulator and its interactive debugger are in Chapter 3. That chapter also holds a placeholder for the planned hardware reference, which will document the intended silicon design.

1 Instruction Set Architecture

This chapter describes the MISA instruction set architecture, include the base instruction set, optional assembler extensions, special registers and hardware data structures, and other implementation-agnostic features.

The MISA architecture is an 8-bit architecture with up to a 16-bit address bus to interface with externally connected memory. Instructions are fixed-width 16-bit doublewords in little-endian byte order.

The information presented in this chapter is referred to as the MISA core architecture. The last section of this chapter enumerates several optional extensions to the core architecture, although an implementation is considering conforming if it implements at least the core architecture.

1.1 General Purpose Registers

The general-purpose register file contains sixteen, 8-bit registers for general computation and data storage, defined by the following ABI:

ABI Name	ABI Purpose	Wide ABI Name	Encoding
RO	Hard-wired zero	N/A	0000
RA	Argument/temporary	RAB	0001
RB	Argument/temporary		0010
RC	Argument/temporary	RCD	0011
RD	Argument/temporary		0100
RE	Argument/temporary	REF	0101
RF	Argument/temporary		0110
RU	Saved	RUV	0111
RV	Saved		1000
RW	Saved	RWX	1001
RX	Saved		1010
RY	Saved	RYZ	1011
RZ	Saved		1100
RT	Argument/return/temporary	N/A	1101
RSCRATCH0	Assembler scratch	RSCRATCH	1110
RSCRATCH1	Assembler scratch		1111

The **R0** register must be hard-wired to ignore writes and always return the value **0b00000000** on reads. Wide registers are formed by two general-purpose registers, which must be equivalent to using the two 8-bit register names in order. For instance, **RAB** is semantically equivalent to **RA**, **RB** in that order. General purpose registers can be taken as the operands to most instructions which operate on registers.

Argument and return registers are **RA** - **RF** and **RT**. The **RT** register should be used as the first return register, if the value being returned logically fits in a single (not wide) register. Otherwise, the register pair **RAB** should be preferred as the first return register. Argument registers are also temporary registers (caller-saved). **RU** through **RZ** are saved registers (callee-saved).

The **RSCRATCH** wide register is reserved as a “scratch” 16-bit register for the assembler. Certain pseudo-instructions and other macros will use this register when a temporary value is needed (like loading an address to jump to). As such, the value of **RSCRATCH** should not be assumed to persist over time.

1.2 Control and Status Registers

Also required are a few special registers, which cannot be used as instruction operands, but can be read and written from general purpose registers with the **RSR** and **WSR** instructions. Special registers are generally wider 16-bit registers that are required for the operation of certain instructions.

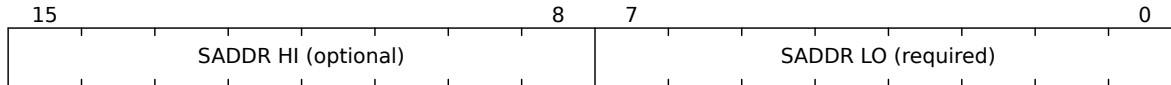
When used as operands for instructions, the special registers are encoded as follows:

ABI Name	Encoding
SADDR	0001
RADDR	0010
FLAGS	0011
CAUSE	0100
EXTNS	0101
RETSC	1000 (under System Call Extension)
PRIVS	1010 (under Privilege Extension)

The following sub-sections describe the purpose and layout of each special register.

1.2.1 SADDR

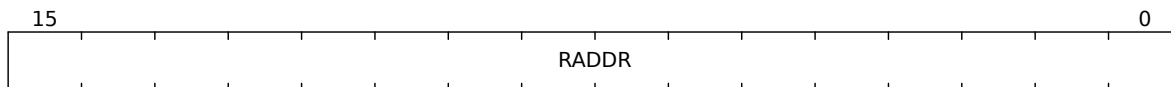
The stack address or stack pointer. **SADDR** must be at least an 8-bit register (supporting stack addresses **0x00** - **0xFF** with a possible high 8-bit prefix) and no more than a 16-bit register.



1.2.2 RADDR

The return address is also a dedicated special register given the limited number of general purpose registers available. **RADDR** is required to be wide enough to store an absolute address for the connected memory, which generally means 16-bit.

The return address special register is not used by specific base instructions, but provides a dedicated 16-bit-wide register for subroutine calling. The **JAL** instruction will set the **RADDR** register with the address of the instruction after the **JAL**. Returning to the location in **RADDR** requires a **RSR** and **JMP**.



1.2.3 FLAGS

Conditional branching in the MISA architecture is based on ALU flag values for comparing integers. Both branching instructions, **JAL** and **JMP**, use these flags to make decisions on whether a branch will be taken.

FLAGS is set based on the result of each ALU operation. The instructions which must update the **FLAGS** register are: **ADD**, **ADC**, **SUB**, **SBB**, **RRC**. Instructions which may, depending on the implementation, update the **FLAGS** register are: **HALT**, **AND**, **OR**, **XOR**. Instructions which must not update the **FLAGS** register are: **SET**, **LD**, **ST**, **RSR**, **WSR** (must not update **FLAGS** asynchronously through the ALU output, but will update **FLAGS** if writing directly to it), **JAL**, **JMP**. A conditional branch that relies on **FLAGS** should be performed immediately after the conditional comparison or a **WSR** to **FLAGS**.

The **FLAGS** special register is a flags register (one bit per flag) which must be at least 4 bits and support the following:

Flag	Bit	Commentary
Z	0	Whether the last ALU operation's output was zero.
C	1	The adder carry out bit of the last ALU operation.
N	2	Whether the last ALU operation's output was negative (MSB = 1).
V	3	Whether the last ALU operation's output overflowed (sign inversion).

1.2.4.1 CAUSE Reason: None

The **CAUSE** register does not represent an exception, or represents an unknown/generic exception without a reason. If the reason is 000, the Type and Extended status fields have no significance.

1.2.4.2 CAUSE Reason: Instruction Usage

The exception cause is related to the use of a specific instruction or class of instructions. The valid Type fields are:

Type	Encoding	Commentary
Halt instruction	000	The HALT instruction was executed. Extended status is the value of the RS operand register. Under the System Call Extension, this reason indicates that the SYSCALL instruction was executed.
Illegal instruction	001	An ill-formed instruction failed to decode. Extended status is not used.
Privileged instruction	010	A privileged instruction was executed while in User Mode (under Privilege Extension).

1.2.4.3 CAUSE Reason: Memory Usage

The exception cause is related to the use of an address or memory operation. The valid Type fields are:

Type	Encoding	Commentary
Address overflow	000	A memory operation attempted to access an address which is outside the mapped memory space.
PC overflow	001	The program counter overflowed an unsigned 16-bit value.
Privileged access	010	An access to privileged memory was attempted while in User Mode (under Privilege Extension).

1.2.5 EXTNS

The **EXTNS** special register is a flags register that holds information about which (if any) architectural extensions are supported by the hardware. All implementations of the MISA architecture must include the **EXTNS** register, even those which only implement the core architecture. A core-only implementation can be identified by **EXTNS == 0x0000**.

Unlike most special registers, writes to **EXTNS** may be ignored (i.e. the register may have a hard-coded value). For more information on writes to **EXTNS**, see the Dynamic Hardware Extension. Note that bit 0 (the Dynamic Hardware Extension) *must* ignore writes regardless of its state, preventing the Dynamic Hardware Extension from disabling itself.

EXTNS must support at least the following bits. A recommended width is 16-bits (with unsupported extensions being forced to 0), but the actual register width is implementation-defined. Unused bits must always be read as zero and ignored on writes (if supported):

Bit	Extension
0	Dynamic Hardware Extension
1	System Call Extension
2	Privilege Extension

1.3 Data Formats and Alignments

The MISA architecture defined in this chapter only supports 8-bit signed integers using two's complement encoding, where the value of an integer is given by the sum of its active bits:

7	6	5	4	3	2	1	0
-127	64	32	16	8	4	2	1

16-bit data types may be supported in software using the **ADC** and **SBB** instructions for arithmetic.

1.4 Memory

The MISA architecture supports up to a 16-bit address bus for memory operations, and memory must be byte-addressable. While the memory map for applications using the MISA architecture is generally free for the user to decide, this section will cover both a recommended map and specific regions which are required by all maps.

1.4.1 Required Memory Regions

The region 0xFFFF0 - 0xFFFFF is reserved for a reset vector table. When the processor is taken out of reset, the program counter must be initialized to the address stored in memory at 0xFFFFE - 0xFFFFF and begin execution there.

The reset vector table contains the following functions:

Vector Address	Purpose
0xFFFF0 - 0xFFFF1	Reserved
0xFFFF2 - 0xFFFF3	Reserved
0xFFFF4 - 0xFFFF5	Reserved
0xFFFF6 - 0xFFFF7	Reserved
0xFFFF8 - 0xFFFF9	Reserved
0xFFFFA - 0xFFFFB	Syscall handler (under System Call Extension)
0xFFFFC - 0xFFFFD	Fault handler
0xFFFFE - 0xFFFFF	Reset handler

1.4.2 Recommended Memory Map

The linker script recommended by default is as follows:

```
// Zero page, which may have fast access on some hardware
SECTION 0x0000 - 0x00FF : direct ;

// Stack, SADDR initialized to 0x(01)FF and decrements
SECTION 0x0100 - 0x01FF : ;

// RAM, may support software-implemented stack or heap
SECTION 0x0200 - 0x7FFF : data bss ;

// Memory-mapped IO and IO buffers
SECTION 0x8000 - 0xBFFF : ;

// Program ROM
SECTION 0xC000 - 0xFFFF : text rodata ;

// Vector table
SECTION 0xFFFF0 - 0xFFFFF : vectors ;
```

Which define the following sections by convention:

Section	Commentary
direct	Contents to place in the first 256-byte page. This page may support faster memory operations on certain hardware implementations, and thus may be a desirable target for frequently accessed data.
data	Initialized global data that is mutable.

Section	Commentary
bss	Declared but uninitialized global data that is mutable. Runtime/startup code should initialize bss symbols to zero.
text	Executable and immutable program code.
rodata	Initialized global constants.
vectors	A small region for reset vector function addresses.

1.4.3 Stack Conventions

The MISA ISA requires a towards-lower-address stack that grows towards 0x0000 on pushes and contracts towards 0xFFFF on pops. This convention must be respected regardless of the supported width of the **SADDR** special register or the location in memory of the stack.

Stack operations must respect the convention of the **PUSH** and **POP** assembler macros. That is, pushing to the stack must write before decrementing **SADDR**, and popping from the stack must increment **SADDR** before reading.

Multi-byte pushes and pops must be logically equivalent to chaining several **PUSH** or **POP** assembler macros.

1.5 Reset

When the processor is taken out of the reset state, it is guaranteed by the ISA to initialize some internal state, including:

- The reset vector address stored in memory at 0xFFFE - 0xFFFF must be read and used to initialize the program counter.

The state of general purpose registers and control and status registers is implementation defined.

1.6 Faults

Fault conditions occur when something goes wrong program execution in the processor. This includes:

- Bad instruction decodes
- Overflow of the program counter
- Etc.

When a fault is detected, the processor must set the **CAUSE** register based on the type of fault and then set the program counter to the fault vector address stored in memory at 0xFFFC - 0xFFFD. Execution must then continue in the fault handler, which is software-defined and may attempt recovery, dump diagnostic information, or simply halt.

1.7 Exceptions and Interrupts

The core ISA defined in this chapter does not support traditional exceptions or interrupts. The **CAUSE** special register is intended for use as a diagnostic tool in the core architecture, and **HALT** is the only instruction that is required to update it.

A basic synchronous exception system is supported under the System Call Extension which, when combined with the Privilege Extension, can be used to invoke privileged routines from the unprivileged user mode.

1.8 Base Instructions

Only 16 base instructions (with distinct opcodes and behavior) are supported by the MISA architecture. This is a hard limit, and the 8-bit MISA architecture does not allow for more than 4 bits of opcode in each instruction word. This subsection defines all the base instructions and their behavior on the microprocessor's state.

1.8.1 HALT

Usage: **HALT** **RS**

Meaning: **CAUSE** = **RS** << 8 | 0x01 ; **exit**(**RS**)

Description: Halt the processor and emit the contents of **RS** for debug.

Encoding:

15	8	7	4	3	2	1	0
Unused							
RS							
0							
0							
0							
0							

1.8.2 ADD

Usage: ADD RD RS1 RS2

Meaning: $RD = RS1 + RS2$

Description: Add the content of RS1 and RS2 and store the result in RD.

Encoding:

15	12	11	8	7	4	3	2	1	0			
RS2				RS1			RD		0	0	0	1

1.8.3 ADC

Usage: ADC RD RS1 RS2

Meaning: $RD = RS1 + RS2 + C$

Description: Add the contents of RS1, RS2, and the C field of the FLAGS register and store the result in RD.

Encoding:

15	12	11	8	7	4	3	2	1	0			
RS2				RS1			RD		0	0	1	0

1.8.4 SUB

Usage: SUB RD RS1 RS2

Meaning: $RD = RS1 - RS2$

Description: Subtract the contents of RS2 from RS1 and store the result in RD.

Encoding:

15	12	11	8	7	4	3	2	1	0			
RS2				RS1			RD		0	0	1	1

1.8.5 SBB

Usage: SBB RD RS1 RS2

Meaning: $RD = RS1 - (RS2 + C)$

Description: Subtract the contents of **RS2** and the **C** field of the **FLAGS** register, as one partial sum, from the contents of **RS1**, and store the result in **RD**.

Encoding:

15				12		11		8			7		4		3		2		1		0				
RS2						RS1						RD						0		1		0		0	

1.8.6 AND

Usage: AND RD RS1 RS2

Meaning: $RD = RS1 \& RS2$

Description: Compute the bitwise AND of the contents of **RS1** and **RS2** and store the result in **RD**.

Encoding:

15	12	11	8	7	4	3	2	1	0			
RS2				RS1			RD		0	1	0	1

1.8.7 OR

Usage: OR RD RS1 RS2

Meaning: $RD = RS1 | RS2$

Description: Compute the bitwise OR of the contents of **RS1** and **RS2** and store the result in **RD**.

Encoding:

15	12	11	8	7	4	3	2	1	0
RS2		RS1		RD		0	1	1	0

1.8.8 XOR

Usage: XOR RD RS1 RS2

Meaning: $RD = RS1 \oplus RS2$

Description: Compute the bitwise exclusive OR of the contents of RS1 and RS2 and store the result in RD.

Encoding:

15				12		11		8			7		4			3		2		1		0			
RS2						RS1						RD						0		1		1		1	

1.8.9 RRC

Usage: RRC RD RS

Meaning: $RD = (C \ll 7) \mid (RS \gg 1)$; $C = RS \& 1$

Description: Shift the 9-bit integer formed by {C, RS} right by one bit. Store the shifted result in RD and the LSB of RS into C. The ALU flags are updated from the 8-bit result. Z is set when the result is zero, N takes the most significant bit of the result, and V is cleared. C receives the shifted-out LSB of RS as described above.

Encoding:

15	12	11	8	7	4	3	2	1	0
Unused		RS		RD		1	0	0	0

1.8.10 SET

Usage: SET RD IMM

Meaning: $RD = IMM$

Description: Store the provided 8-bit immediate value into RD.

Encoding:

15						8	7			4	3	2	1	0
IMM							RD				1	0	0	1

1.8.11 LD

Usage: LD RD RS1 RS2

Meaning: $RD = \text{Memory}[RS1 \ll 8 \mid RS2]$

Description: Load the 8-bit word at the address formed by {RS1, RS2} into RD.

Encoding:

15	12	11	8	7	4	3	2	1	0
RS2				RS1			RD		1010

1.8.12 ST

Usage: ST RD RS1 RS2

Meaning: $\text{Memory}[RS1 \ll 8 \mid RS2] = RD$

Description: Store the 8-bit word in RD to the address formed by {RS1, RS2}.

Encoding:

15	12	11	8	7	4	3	2	1	0
RS2				RS1			RD		1011

1.8.13 RSR

Usage: RSR RS1 RS2 CSR

Meaning: $RS1 = CSR \gg 8$; $RS2 = CSR \& 0xFF$

Description: Read the 16-bit control and status register CSR into the two 8-bit general purpose registers {RS1, RS2}

Encoding:

15	12	11	8	7	4	3	2	1	0
CSR				RS2			RS1		1100

1.8.14 WSR

Usage: WSR RS1 RS2 CSR

Meaning: $CSR = RS1 \ll 8 \mid RS2$

Description: Write the contents formed by {RS1, RS2} into the 16-bit control and status register CSR.

Encoding:

15	12	11	8	7	4	3	2	1	0
CSR				RS2			RS1		
							1	1	0
									1

1.8.15 JAL

Usage: JAL CMP RS1 RS2

Meaning: if (CMP) { RADDR = PC + 2 ; PC = $RS1 \ll 8 \mid RS2$ }

Description: If the comparison type CMP is true as determined by the current ALU flags, set RADDR to the address of the next instruction and set PC to the address formed by {RS1, RS2}.

Encoding:

15	12	11	8	7	4	3	2	1	0
CMP				RS2			RS1		
							1	1	1
									0

1.8.16 JMP

Usage: JMP CMP RS1 RS2

Meaning: if (CMP) { PC = $RS1 \ll 8 \mid RS2$ }

Description: If the comparison type CMP is true as determined by the current ALU flags, set PC to the address formed by {RS1, RS2}.

Encoding:

15	12	11	8	7	4	3	2	1	0
CMP				RS2			RS1		
							1	1	1
									1

1.9 Pseudo Instructions

Because of the limitations of the core instruction set, this document also recommends a set of assembler macros whose functionality can be represented by one or more of the base 16 instructions.

1.9.1 ADD2

Usage: ADD2 RD1 RD2 RS1 RS2 RS3 RS4

Meaning: ADD RD2 RS2 RS4, ADC RD1 RS1 RS3

Description: Add the 16-bit values {RS1, RS2} and {RS3, RS4} and store the result in {RD1, RD2}.

1.9.2 AND2

Usage: AND2 RD1 RD2 RS1 RS2 RS3 RS4

Meaning: AND RD2 RS2 RS4, AND RD1 RS1 RS3

Description: Compute the bitwise AND of the contents of {RS1, RS2} and {RS3, RS4} and store the result in {RD1, RD2}

1.9.3 CALL

Usage: CALL IMM

Meaning: SET RSCRATCH0 (IMM >> 8), SET RSCRATCH1 (IMM & 0xFF), JAL ALWAYS RSCRATCH0 RSCRATCH1

Description: Jump to the subroutine at the address IMM and link with the RADDR special register.

1.9.4 CLR

Usage: CLR

Meaning: WSR FLAGS R0 R0

Description: Set the FLAGS special register to zero, effectively lowering all flags.

1.9.5 CMP

Usage: CMP RS1 RS2

Meaning: SUB R0 RS1 RS2

Description: Perform an arithmetic operation which will set the **FLAGS** register such that the boolean operation ? can be performed on RS1 ? RS2.

1.9.6 GOTO

Usage: GOTO IMM

Meaning: SET RSCRATCH0 (IMM >> 8), SET RSCRATCH1 (IMM & 0xFF), JMP ALWAYS RSCRATCH0 RSCRATCH1

Description: Unconditionally jump to the address IMM.

1.9.7 JALI

Usage: JALI CMP IMM

Meaning: SET RSCRATCH0 (IMM >> 8), SET RSCRATCH1 (IMM & 0xFF), JAL CMP RSCRATCH0 RSCRATCH1

Description: If the comparison type **CMP** is true as determined by the current ALU flags, set **RADDR** to the address of the next instruction and set **PC** to the address **IMM**.

1.9.8 JMPI

Usage: JMPI CMP IMM

Meaning: SET RSCRATCH0 (IMM >> 8), SET RSCRATCH1 (IMM & 0xFF), JMP CMP RSCRATCH0 RSCRATCH1

Description: If the comparison type **CMP** is true as determined by the current ALU flags, set **PC** to the address **IMM**.

1.9.9 MOV

Usage: MOV RD RS1

Meaning: OR RD RS1 R0

Description: Copy the contents of RS1 into RD.

1.9.10 MOV2

Usage: MOV2 RD1 RD2 RS1 RS2

Meaning: MOV RD2 RS2, MOV RD1 RS1

Description: Copy the contents of the 16-bit value {RS1, RS2} into {RD1, RD2}.

1.9.11 NOP

Usage: NOP

Meaning: OR R0 R0 R0

Description: Do nothing.

1.9.12 OR2

Usage: OR2 RD1 RD2 RS1 RS2 RS3 RS4

Meaning: OR RD2 RS2 RS4, OR RD1 RS1 RS3

Description: Compute the bitwise OR of the contents of {RS1, RS2} and {RS3, RS4} and store the result in {RD1, RD2}.

1.9.13 POP

Usage: POP RD

Meaning:

```
RSR SADDR RSCRATCH
SET RD 1
ADD RSCRATCH1 RSCRATCH1 RD
ADC RSCRATCH0 RSCRATCH0 R0
LD RD RSCRATCH
WSR SADDR RSCRATCH
```

Description: Increment SADDR by 1 byte and then load the byte at the new stack address into RD.

1.9.14 POP2

Usage: POP2 RD1 RD2

Meaning: POP RD1, POP RD2

Description: Pop a 16-bit value off the stack and into {RD1, RD2}.

1.9.15 PUSH

Usage: PUSH RS

Meaning:

```
RSR SADDR RSCRATCH
ST RS RSCRATCH
ADD RO RO RO
SBB RSCRATCH1 RSCRATCH1 RO
SBB RSCRATCH0 RSCRATCH0 RO
WSR SADDR RSCRATCH
```

Description: Write RS onto the stack at SADDR, and then decrement SADDR by 1 byte.

1.9.16 PUSH2

Usage: PUSH2 RS1 RS2

Meaning: PUSH RS2, PUSH RS1

Description: Push a 16-bit value onto the stack from {RS1, RS2}.

1.9.17 RET

Usage: RET

Meaning: RSR RADDR RSCRATCH, JMP ALWAYS RSCRATCH

Description: Return to the address in RADDR.

1.9.18 RRC2

Usage: RRC2 RD1 RD2 RS1 RS2

Meaning: RRC RD1 RS1, RRC RD2 RS2

Description: Shift the 17-bit integer formed by {C, RS1, RS2} right by one bit. Store the shifted result in {RD1, RD2} and the LSB of RS2 into C.

1.9.19 SET2

Usage: SET2 RS1 RS2 IMM

Meaning: SET RS1 (IMM >> 8), SET RS2 (IMM & 0xFF)

Description: Set both the RS1 and RS2 registers from the high and low halves of a 16-bit immediate value.

1.9.20 SUB2

Usage: SUB2 RD1 RD2 RS1 RS2 RS3 RS4

Meaning: SUB RD2 RS2 RS4, SBB RD1 RS1 RS3

Description: Subtract the 16-bit value {RS3, RS4} from {RS1, RS2} and store the result in {RD1, RD2}.

1.9.21 XOR2

Usage: XOR2 RD1 RD2 RS1 RS2 RS3 RS4

Meaning: XOR RD2 RS2 RS4, XOR RD1 RS1 RS3

Description: Compute the bitwise XOR of the contents of {RS1, RS2} and {RS3, RS4} and store the result in {RD1, RD2}

1.10 Architecture Extensions

The content in this chapter until this point is the MISA core architecture. This section provides a description of several optional extensions to that architecture which enable additional functionalities.

1.10.1 Dynamic Hardware Extension

The Dynamic Hardware Extension enables writes to the **EXTNS** special register and thus dynamic support for the architecture extensions discussed in this section.

Note: If the Dynamic Hardware Extension is supported, writes to the Dynamic Hardware Extension bit of the **EXTNS** special register must always be ignored. The extension cannot be used to disable itself at runtime.

If the Dynamic Hardware Extension is not supported, writes to any bit of **EXTNS** are ignored. The list of supported extensions is static and a property of the hardware itself. If supported, writes must take affect on the instruction following the **WSR**.

1.10.2 System Call Extension

The System Call Extension extends the **HALT** instruction in the core architecture to support basic synchronous exceptions that may be used for system calls and other similar features.

1.10.2.1 SYSCALL

The **SYSCALL** instruction replaces the **HALT** instruction in the assembler under the System Call Extension. It's usage is similar to **HALT**, but with a conditional “exception” bit that controls the terminal behavior of the processor.

Usage: **SYSCALL** **RS**

Meaning:

CAUSE = **RS** << 8 | 0x01

if (**E** == 0) { exit(**RS**) } else { **PC** = Memory[0xFFFFB] << 8 | Memory[0xFFFFA] }

Encoding:

15	14					8	7			4	3	2	1	0
E					Unused				RS		0	0	0	0

1.10.2.2 HALT

HALT functionality as defined in the core architecture is supported under the System Call Extension by setting the **E** bit to 0. In that case, the processor must adhere to the core architecture behavior for **HALT**, including stopping the clock and setting the **CAUSE** register.

1.10.2.3 Calling Conventions and Stack Frame

When a **SYSCALL** instruction is executed, the contents of the **RS** register are used as the syscall code, stored in the **CAUSE** special register. The syscall handler address at **0xFFFFA - 0xFFFFB** is responsible for reading the **CAUSE** register, understanding that a syscall was executed, and using the extended status value to branch to different handler code for each type of call.

General purpose registers follow the same caller-callee conventions as standard subroutine/function calls in the MISA ABI. This includes which registers are used as arguments to the system call, and which registers are used as return values from the system call. All special registers must be saved and restored by the system call routine.

1.10.2.4 Nested System Calls

The **SYSCALL** instruction may not be executed within a system call context. The processor is canonically in a system call context under the following formal definition:

- Let the initial **SYSCALL** instruction be at $PC = N$
- Let the return address of the **SYSCALL** be at $PC = N + 2$
- The processor is in a system call context for all instructions between the **SYSCALL** at $PC = N$ (exclusive) and the return instruction at $PC = N + 2$ (exclusive)

If a secondary **SYSCALL** is performed in a system call context, a fault must occur with the **CAUSE** Reason “Instruction usage” and the **CAUSE** Type “Illegal instruction”.

1.10.2.5 RETS

No instruction is explicitly reserved for returning from a system call-based exception, and the exact return process is implementation-defined provided it adheres to the conventions described above.

The canonical implementation defines and uses the **RETSC** special register. When a **SYSCALL** instruction is executed, the return address of the system call is stored in **RETSC**.



The following pseudo-instruction is defined to return to this address:

Usage: **RETS**

Meaning: **RSR RETSC RSCRATCH, JMP ALWAYS RSCRATCH**

Description: Return from a system call to the address in **RETSC**.

1.10.3.3 Points of Privilege Change

Under normal operation, the processor is expected to change privilege modes when the following occur:

- When any fault occurs which requires the processor to jump to the fault handler (including an User Mode privilege error), the processor must first set `PRIVS.R` to 0.
- Under the System Call Extension, when the `SYSCALL` instruction is executed and the exception bit is 1, `PRIVS.R` must be set to 0 before the system call handler is entered.
- Under the System Call Extension, when the processor is returned from a system call context, the `PRIVS.R` bit must be restored to its initial value before the `SYSCALL`.
- When `WSR` is executed on the `PRIVS` special register from Machine Mode, the updated ring bit must take affect for the next instruction.

2 Toolchain

The MISA project provides a standard toolchain for low-level development of programs on a processor supporting the full MISA architecture. This includes an assembler, linker, some debug/analysis tools, and a reference simulator.

This chapter covers the usage and technical details of the toolchain for building programs. That is, all components of the MISA software package except the functional simulator.

2.1 Assembler

`misa-as` is the reference assembler for the MISA instruction set architecture described in Chapter 1 of this document. It supports all the behavior of the ISA chapter, including the recommended pseudo instructions and assembler macros.

Using `misa-as` is similar to an abbreviated version of the GNU `as` assemblers and others like it. For a full description, see the help text at `misa-as --help`. In short, there are two common ways to use the assembler:

- 1) Assembling source files to a binary: `misa-as file1.S file2.S ... fileN.S`
- 2) Assembling source files to object files: `misa-as -a file1.S file2.S ... fileN.S`

The assembler produces object files in a simple format accepted by the MISA linker. By default, assembling will also invoke the linker to produce a final binary, although this step can be omitted.

2.1.1 Assembler Pipeline

The `misa-as` assembler is a wrapper CLI utility for several core binaries including the assembler, but also the preprocessor and linker.

Assembling with no early-exit flags (e.g. `-p` or `-a`) runs provided files through a multi-step pipeline:

- 1) All assembly source files are passed to the MISA preprocessor to resolve any preprocessor directives. The pipeline can be stopped after this stage with the `-p` flag.

- 2) The preprocessed output files are passed to the MISA assembler, which produces object files. The pipeline can be stopped after this stage with the `-a` flag.
- 3) The generated object files, as well as any non-source files originally passed to `misa-as`, are passed to the MISA linker to produce the final flat binary.

2.1.2 Assembler Semantics

2.1.2.1 Instructions

`misa-as` supports the instruction semantics defined in the instruction “Usages” of the ISA chapter. Opcodes and instruction operands are not comma-separated, and are case-insensitive.

2.1.2.2 Labels

Labels are programmer-defined symbols in assembly source code that are resolved to specific addresses by the linker. Labels are C-style identifiers (containing the characters ‘_’, a-z, A-Z, and 0-9, but not starting with 0-9) which end with a colon (‘:’).

Label names are global to a binary at the linking stage, and can only be defined once. Assembly written for the MISA standard library and other system code reserves double-underscore prefix `__` for “local” labels. The recommended convention for local labels created by compilers and non-library assembly is a single-underscore prefix `_`. In both cases, local labels should use the name of their containing function as a discriminant. Compilers are encouraged to support name mangling for global function labels.

During linking, labels used as instruction operands will be resolved to the address of the first instruction after the label. For example, consider the following code:

```
    SET RA 0
    SET RB 10
loop:
    SET2 RYZ loop
    JMP LESS RA RB
```

In this case, the label `loop` has the relative address in this code of `0x0004`, and refers to the `SET2` instruction.

2.1.2.3 Assembler Directives

`misa-as` supports several assembler directives that change the behavior of the assembler and its code generation. Directives are C-style reserved identifiers that begin with a period (.) and may take one or more operands.

The following directives are currently supported by `misa-as`:

- `.word <Word8>`: Emits the raw byte `<Word8>` in the generated code at the location of the directive
- `.array <Word8> [Word8...]`: Emits the raw bytes `<Word8> [Word8...]` in the generated code at the location of the directive. Earlier bytes in the array definition will be placed at lower addresses. `.array` is equivalent to many consecutive `.word` directives
- `.addr <Label>`: Emits the two-byte address of the label `<Label>` after linking at the location of the directive. If the linker places `<Label>` at `<HiByte> - <LoByte>`, this directive is equivalent to `.word <LoByte> .word <HiByte>`.
- `.ascii "<String>"`: Emits the bytes of the string `<String>` using the ASCII encoding at the location of the directive. The first letter of the string is placed at a lower address. The string is not null-terminated. Escape characters are not supported.
- `.asciiz "<String>"`: Equivalent to `.ascii "<String>"` followed immediately by `.word 0x00`. That is, the string is null-terminated with a zero byte at the higher address following the last character of the string.
- `.space <Word16>`: Reserves a block of memory of length `<Word16>`. The contents of the memory is undefined at runtime. Linker implementations may either fill the memory or simply not place other data/text in the reserved area, leaving it uninitialized.
- `.section <String>`: Defines for the linker that all the following code until the next section directive should be placed in a section called `<String>`

2.1.2.4 Preprocessor Directives

`misa-as` also supports preprocessor directives, but these are not resolved by the assembler directly. See the section of the MISA preprocessor `misa-pr` for more information on syntax. Preprocessor directives are resolved by the assembler before the assembly or linking phases.

2.1.3 MISA Linkable Format

Object files produced by `misa-as` are in the custom `MISA_LF` linkable format, which are binary files that contain:

- The assembled source code for one or more sections defined in a single assembler translation unit.

- A list of symbols (assembler labels) defined in the translation unit, along with their addresses relative to the beginning of the object file's machine code.
- A list of symbols required (not defined) by the translation unit during the linking phase, along with the address relative to the beginning of the object file's machine code where the symbol is needed.

2.1.3.1 Object File Description

Binary object files and all fields within them are encoded with little endian byte order. Each object file contains the following information:

Length	Type	Content
8	String	The string <code>MISA_LF□</code> , where M is the least significant byte and □ is the most significant byte.
2	Word16	The number of sections present in the object file.
Variable	[Sec]	The content of each section, repeated as many times as specified in the length field.
2	Word16	The number of strings present in the string table.
Variable	[(Word16, String)]	The ordered list of all string identifiers preceeded by their lengths, repeated as specified in the length field.

2.1.3.2 Object File Section Description

Each section contains the following information:

Length	Type	Content
2	Word16	The length in bytes of the section name string field.
Variable	String	The name of the section as defined with the <code>.section</code> assembler directive.
Variable	Code	The content of the assembled machine code for the specified section.
Variable	SymTable	The table of symbols defined by the code in the specified section.
Variable	RelocTable	The table of relocations required by the code in the specified section.

2.1.3.3 Object File Code Description

Each code table contains the following information:

Length	Type	Content
2	Word16	The length in bytes of the machine code.
Variable	[Word8]	The machine code.

2.1.3.4 Object File Symbol Table Description

Each symbol table contains the following information:

Length	Type	Content
2	Word16	The number of symbols in the symbol table.
Variable	[Sym]	The list of symbols.

Each entry in the symbol table contains the following information:

Length	Type	Content
2	Word16	The address relative to the start of the code table that the symbol points to.
2	Word16	The 0-aligned index into the object file's string table where the symbol's name can be found.

2.1.3.5 Object File Relocation Table Description

Each relocation table contains the following information:

Length	Type	Content
2	Word16	The number of relocations in the relocation table.
Variable	[Reloc]	The list of relocations.

Each entry in the relocation table contains the following information:

Length	Type	Content
1	Word8	Whether the byte at the relocation address must be filled with the high (0x01) or low (0x00) half of the relocated symbol's address.
2	Word16	The address relative to the start of the code table of the byte that needs to be filled by the linker.

Length	Type	Content
2	Word16	The 0-aligned index into the object file's string table where the symbol's name can be found.

2.1.3.6 Object File Example

Consider the following code stub that has code, exported symbols, and required relocations:

```
.section text
_start:           // _start is a symbol
    SET2 RSCRATCH 0x01FF
    WSR SADDR RSCRATCH
    CALL main      // main is a relocation
    HALT RT
```

Assembling this code to an object file with `misa-as -a start.S -o start.o` will produce the following object code:

00000000:	4d49 5341 5f4c 4620 0100 0400 7465 7874	MISA_LF ...text
00000010:	0e00 e901 f9ff ed1f e900 f900 ee0f d000	
00000020:	0100 0000 0000 0200 0107 0001 0000 0900	
00000030:	0100 0200 0600 5f73 7461 7274 0400 6d61	_start..ma
00000040:	696e	in

Header, # of sections Symbol table
Section name Relocation table
Code segment String table

2.2 Linker

`misa-ld` is the reference linker for the MISA_LF linkable format of object files. It uses memory map files to place the code sections of one or more object files into a flat binary, resolving symbol positions along the way.

2.3 Other Tools

3 Simulator And Hardware Reference

This chapter covers the reference simulator for the MISA architecture and the planned hardware implementation.

3.1 Functional Simulator

`misa-sim` is the functional reference implementation of the MISA instruction set. It executes the flat binaries produced by the linker and models the processor as a single-cycle machine. The simulator is intended to match Chapter 1 exactly, so it serves as an executable reference for the behavior of every instruction and how those instructions interact with memory, processor state, etc.

The simulator comes with an interactive GDB-style debugger for loading, inspecting, and executing programs. The debugger is documented in the next section.

3.1.1 Loading And Reset

A program is loaded into memory as a flat binary starting at a byte offset, which defaults to `0x0000`. Loading copies bytes from the file into memory until either the file ends or the top of memory is reached. The linker emits an image that spans the whole address space including the vector table, so a linked binary is normally loaded at offset `0x0000`.

The simulator resets after a binary is loaded. Reset reads the reset vector stored in little-endian at `0xFFFFE - 0xFFFFF` and initializes the program counter to that address. The processor is then ready to execute instructions.

3.1.2 Instruction Execution

Each step fetches the 16-bit little-endian instruction at the program counter, and advances the program counter by two bytes before executing. Execution of each instruction follows the semantics in Chapter 1. When the program halts, the simulator returns to its reset state and stops.

A faulting instruction discards its partial result, records the reason and type in the `CAUSE` register, and sets the program counter to the fault handler address stored at `0xFFFFC - 0xFFFFD`.

Execution then continues in the fault handler. This matches the fault behavior defined in Chapter 1.

3.1.3 Address Space

The reference simulator implements the entire 16-bit address space. All 65536 bytes are mapped, and memory accesses wrap at 0xFFFF, so an access past the top of memory returns to the bottom. Because no region is unmapped, the simulator never raises the “Address overflow” memory usage fault from Chapter 1. That fault is reserved for implementations with a narrower bus or a memory protection unit. The simulator does still raise the “PC overflow” fault when the program counter would leave the 16-bit range during an increment.

3.2 Interactive Debugger

The simulator is driven by an interactive debugger, started with `misa-sim [binary]`. The optional binary is loaded at startup as though passed to the `load` command. The debugger presents a (`misa`) prompt and reads one command per line. Command history is saved between sessions.

Type `help` to list the available commands, or `help <command>` for the detailed help of a single command. Most commands accept short aliases, listed below. Arguments in angle brackets are required, and arguments in square brackets are optional.

Command	Aliases	Usage	Description
<code>load</code>		<code>load <file> [offset]</code>	Load a binary image into memory at [offset] (default 0x0000) and reset the simulator.
<code>symbol-file</code>	<code>symbol, sym</code>	<code>symbol-file <file></code>	Import a symbol table from an object file for use by later commands.
<code>disassemble</code>	<code>disass</code>	<code>disassemble [start, [stop\ +count]]</code>	Disassemble memory, defaulting to a window from 8 bytes before to 16 bytes after the program counter.
<code>reset</code>		<code>reset</code>	Reset the processor and jump to the reset vector.
<code>step</code>	<code>si, s</code>	<code>step [n]</code>	Execute at most [n] instructions (default 1), stopping early on a halt or breakpoint.
<code>continue</code>	<code>cont, c</code>	<code>continue</code>	Run until a breakpoint or a halt.
<code>breakpoint</code>	<code>break, br, b</code>	<code>breakpoint <addr\ sym></code>	Set a breakpoint at an address or an imported symbol.

Command	Aliases	Usage	Description
<code>delete</code>	<code>del, d</code>	<code>delete [addr\ sym]</code>	Remove one breakpoint, or all breakpoints when given no argument.
<code>info</code>	<code>i</code>	<code>info <topic></code>	Show simulator state. The topics are registers and breakpoints .
<code>examine</code>	<code>exam, x</code>	<code>examine <addr> [count]</code>	Dump [count] bytes of memory (default 16) starting at <addr>.
<code>quit</code>	<code>q</code>	<code>quit</code>	Save history and exit the simulator.

Breakpoints must be placed at the lower byte of an instruction to be hit. A symbol argument to **breakpoint**, **delete**, or **disassemble** is resolved to an address immediately using the most recently imported symbol file, so later imports do not move an existing breakpoint. Disassembly and symbol import both require the referenced files to still exist on disk.

3.3 Hardware Reference

The ultimate goal of the MISA project is to design, verify, and manufacture an ASIC that runs a complete implementation of the architecture. Tapeout shuttle programs that offer such a service often carry restrictive area and pinout requirements, so the intended design is a single-cycle processor sized for those constraints.

The hardware reference is planned work.